影像處理
--CH9

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/12/16

1. text-broken.tif

1）**source code**:

```
import cv2
import numpy as np
import os
import sys
from google.cloud import vision
current_dir = os.path.dirname(os.path.abspath(__file__))
key_path = os.path.join(current_dir, "key.json")

if os.path.exists(key_path):
    print(f"[INFO] Successfully loaded Service Account Key: {key_path}")
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = key_path
else:
    print(f"[ERROR] Key file not found at: {key_path}")
    sys.exit()

def join_broken_text(image_path):
    print(f"[STEP 1] Processing image: {image_path} ...")
    img = cv2.imread(image_path, 0)
    if img is None:
        print(f"[ERROR] Could not read image file. Please check the
filename: {image_path}")
        sys.exit()
    _, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    joined_img = cv2.dilate(binary_img, kernel, iterations=1)
    output_filename = "joined_text_final.png"
    cv2.imwrite(output_filename, joined_img)
    print(f"[INFO] Image processing complete. Saved as:
{output_filename}")
    return output_filename

def recognize_text_google(image_path):
```

```
    print(f"[STEP 2] Uploading to Google Cloud Vision API for
recognition...")
    client = vision.ImageAnnotatorClient()
    with open(image_path, 'rb') as image_file:
        content = image_file.read()
    image = vision.Image(content=content)
    response = client.document_text_detection(image=image)
    if response.error.message:
        raise Exception(f'API Error: {response.error.message}')
    return response.full_text_annotation.text

if __name__ == "__main__":
    input_image = "text-broken.tif"
    if not os.path.exists(input_image):
        print(f"[WARNING] '{input_image}' not found. Searching for
common filenames...")
        if os.path.exists("image_2.png"):
             input_image = "image_2.png"
        elif os.path.exists("joined_text_result.png"):
             input_image = "joined_text_result.png"
        else:
            print("[ERROR] No image file found. Please verify the file
exists and update the 'input_image' variable in the code.")
            sys.exit()
    try:
        processed_image_path = join_broken_text(input_image)
        ocr_result = recognize_text_google(processed_image_path)
        print("\n==============================================")
        print("          FINAL OCR RESULT                    ")
        print("==============================================")
        print(ocr_result)
        print("==============================================")
    except Exception as e:
        print(f"\n[EXCEPTION] An error occurred: {e}")
```

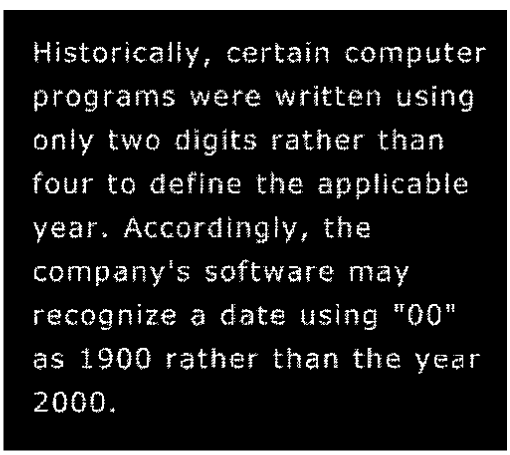
2) describe method:

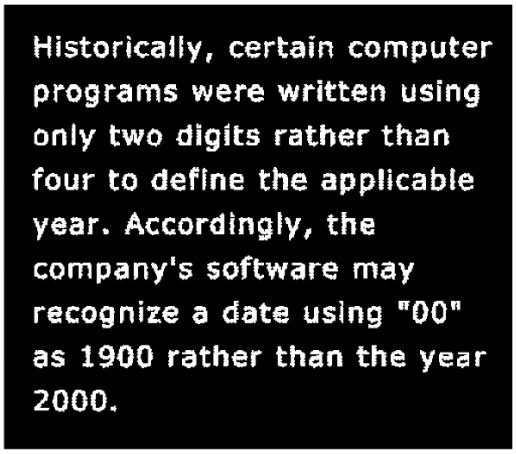
首先，針對原始影像進行前處理，將其轉換為灰階格式並實施二值化閾值處理，藉此濾除不必要的色彩資訊與背景雜訊，使文字與背景形成明確的黑白對比。

接著，採用Morphological Dilation方式來修復文字。定義了一個 3x3 像素的矩形結構元素作為掃描視窗，利用膨脹運算的特性，當視窗覆蓋範圍內偵測到前景像素時，即擴張中心點的像素區域。

最後，將修復完成的影像串接至 Google Cloud Vision API 進行文字辨識。有別於傳統 OCR 引擎易受筆畫變形影響，本實驗採用的文件文字偵測（Document Text Detection）模型具備深度學習架構，能針對密集文本進行特徵提取與上下文分析。

3) Test join result:





Original　　Result

4) OCR result:

```
==============================================
          FINAL OCR RESULT
==============================================
Historically, certain computer
programs were written using
only two digits rather than
four to define the applicable
year. Accordingly, the
company's software may
recognize a date using "00"
as 1900 rather than the year
2000.
==============================================
```

2. text.tif

1) source code:

```
import cv2
import numpy as np
import os
import sys
from google.cloud import vision
current_dir = os.path.dirname(os.path.abspath(__file__))
key_path = os.path.join(current_dir, "key.json")
if os.path.exists(key_path):
    print(f"[INFO] Successfully loaded Service Account Key: {key_path}")
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = key_path
else:
    print(f"[ERROR] Key file not found at: {key_path}")
    sys.exit()

def clear_border_text(image_path):
    print(f"[STEP 1] Processing image borders: {image_path} ...")
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"[ERROR] Could not read image: {image_path}")
        sys.exit()
    _, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    num_labels, labels, stats, centroids =
cv2.connectedComponentsWithStats(binary_img, connectivity=8)
    H, W = binary_img.shape
    cleaned_img = binary_img.copy()
    for i in range(1, num_labels):
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        w = stats[i, cv2.CC_STAT_WIDTH]
        h = stats[i, cv2.CC_STAT_HEIGHT]
        touches_border = (x <= 1) or (y <= 1) or (x + w >= W - 1) or (y
+ h >= H - 1)
        if touches_border:
            cleaned_img[labels == i] = 0
    output_filename = "cleaned_text.tif"
```

```
    cv2.imwrite(output_filename, cleaned_img)
    print(f"[INFO] Image cleaning complete. Saved as:
{output_filename}")
    return output_filename

def recognize_text_google(image_path):
    print(f"[STEP 2] Uploading to Google Cloud Vision API for
recognition...")
    client = vision.ImageAnnotatorClient()
    with open(image_path, 'rb') as image_file:
        content = image_file.read()
    image = vision.Image(content=content)
    image_context = vision.ImageContext(language_hints=["en"])
    response = client.document_text_detection(image=image,
image_context=image_context)
    if response.error.message:
        raise Exception(f'API Error: {response.error.message}')
    return response.full_text_annotation.text

if __name__ == "__main__":
    input_image = 'text.tif'
    if not os.path.exists(input_image):
        print(f"[WARNING] '{input_image}' not found.")
        if os.path.exists("2025-12-13 112047.jpg"):
             input_image = "2025-12-13 112047.jpg"
             print(f"[INFO] Using alternative file: {input_image}")
        else:
            print("[ERROR] No image file found. Please verify the file
exists in the directory.")
            sys.exit()
    try:
        cleaned_image_path = clear_border_text(input_image)
        ocr_result = recognize_text_google(cleaned_image_path)
        print("\n==============================================")
        print("          FINAL OCR RESULT                    ")
        print("==============================================")
        print(ocr_result)
        print("==============================================")
```

```
    except Exception as e:
        print(f"\n[EXCEPTION] An error occurred: {e}")
```

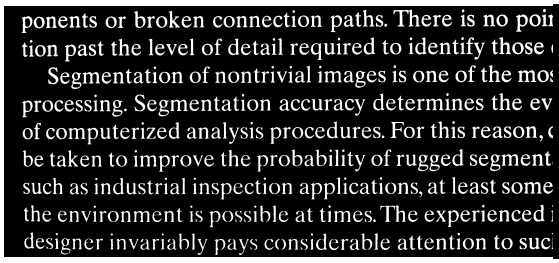
2) describe method:

首先針對輸入的原始影像進行前處理，將其轉換為灰階格式並實施二值化閾值處理。此步驟的目的是將影像單純化為黑與白兩個極端值，設定文字為白色前景、背景為黑色，以便讓電腦能清楚區分出每一個獨立的筆畫或字母結構。在取得二值化影像後，採用 OpenCV 的 Connected Components Analysis來對影像進行掃描。此演算法會將相鄰的白色像素視為同一個物件，並為影像中每一個獨立的文字區塊分配一個唯一的標籤，同時計算出該區塊的邊界框資訊，包含其座標位置（x, y）以及寬度與高度。

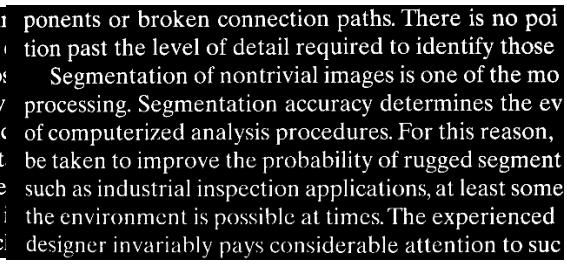
透過程式邏輯逐一檢查每一個文字區塊的邊界框，判斷其是否與影像的四個邊緣（上、下、左、右）發生接觸。只要某個文字區塊的座標位置落在了影像的最外層邊界上，該區塊即被判定為觸碰邊緣的雜訊或不完整的截斷文字。程式隨即將該區塊對應的所有像素值強制歸零（塗黑），將其從影像中完全抹除，最終僅保留下未觸碰邊緣、位於影像中央區域的完整文字內容。

最後是將處理過的乾淨影像串接至 Google Cloud Vision API。由於影像邊緣的殘缺文字已在前一步驟被濾除，這能有效避免 OCR 模型因為讀取到只有一半的字母而產生亂碼或誤判。

3) Clear result:



Original



Result

4) OCR result:

```
================================================
          FINAL OCR RESULT
================================================
ponents or broken connection paths. There is no poi
tion past the level of detail required to identify those
Segmentation of nontrivial images is one of the mo
processing. Segmentation accuracy determines the ev
of computerized analysis procedures. For this reason,
be taken to improve the probability of rugged segment
such as industrial inspection applications, at least some
the environment is possible at times. The experienced
designer invariably pays considerable attention to suc
================================================
```

3. text-sineshade.tif

1) Source code:

```
import cv2
import numpy as np
import os
import sys
from google.cloud import vision
current_dir = os.path.dirname(os.path.abspath(__file__))
key_path = os.path.join(current_dir, "key.json")
if os.path.exists(key_path):
    print(f"[INFO] Successfully loaded Service Account Key: {key_path}")
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = key_path
else:
    print(f"[ERROR] Key file not found at: {key_path}")
    print("[ERROR] Please make sure 'key.json' is in the same folder.")
    sys.exit()

def remove_periodic_noise(image_path):
    print(f"[STEP 1] Processing image: {image_path} ...")
    img = cv2.imread(image_path, 0)
    if img is None:
        print(f"[ERROR] Could not read image file: {image_path}")
        sys.exit()
    dft = cv2.dft(np.float32(img), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)
    rows, cols = img.shape
    crow, ccol = rows // 2, cols // 2
    mask = np.ones((rows, cols, 2), np.uint8)
    r_vertical = 15
    r_horizontal = 10
    mask[crow-r_vertical:crow+r_vertical, ccol-r_horizontal:ccol-1] = 0
    mask[crow-r_vertical:crow+r_vertical, ccol+1:ccol+r_horizontal] = 0
    fshift = dft_shift * mask
    f_ishift = np.fft.ifftshift(fshift)
    img_back = cv2.idft(f_ishift)
    img_back = cv2.magnitude(img_back[:,:,0], img_back[:,:,1])
```

```
    cv2.normalize(img_back, img_back, 0, 255, cv2.NORM_MINMAX)
    img_clean = np.uint8(img_back)
    _, img_binary = cv2.threshold(img_clean, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
    output_filename = "cleaned_image_result.png"
    cv2.imwrite(output_filename, img_binary)
    print(f"[INFO] Noise removal complete. Saved as: {output_filename}")
    return output_filename

def recognize_text_google(image_path):
    print(f"[STEP 2] Uploading to Google Cloud Vision API ...")
    client = vision.ImageAnnotatorClient()
    with open(image_path, 'rb') as image_file:
        content = image_file.read()
    image = vision.Image(content=content)
    response = client.document_text_detection(image=image)
    if response.error.message:
        raise Exception(f'API Error: {response.error.message}')
    full_text = response.full_text_annotation.text
    return full_text
if __name__ == "__main__":
    input_image = "text-sineshade.tif"
    if not os.path.exists(input_image):
        print(f"[ERROR] Input file '{input_image}' not found.")
        sys.exit()
    try:
        clean_image_path = remove_periodic_noise(input_image)
        text_result = recognize_text_google(clean_image_path)
        print("\n==============================================")
        print("          FINAL OCR RESULT                   ")
        print("==============================================")
        print(text_result)
        print("==============================================")

    except Exception as e:
        print(f"\n[EXCEPTION] An error occurred: {e}")
```

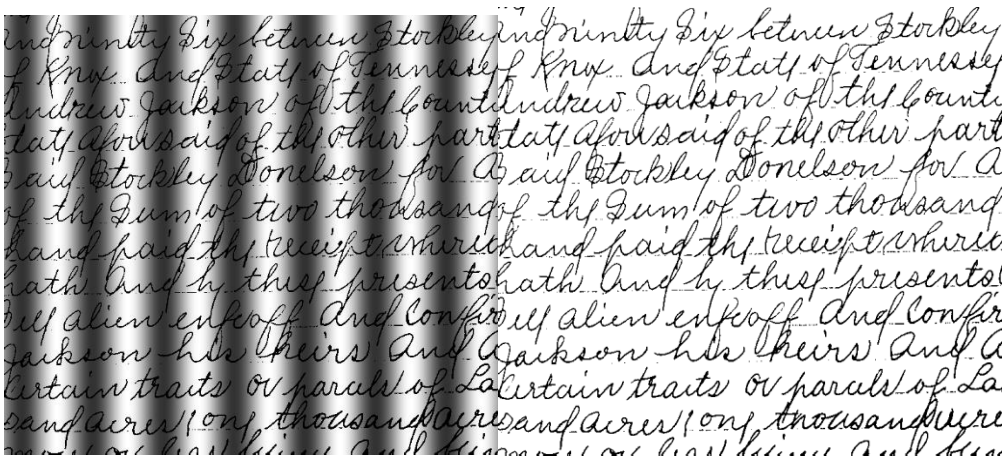
2) Describe method:

首先針對受到正弦波條紋干擾的原始影像進行數位訊號處理。由於這種規律的週期性雜訊在空間域中難以直接透過傳統的模糊或銳化濾波器去除，因此採用離散傅立葉變換將影像從空間域轉換至頻率域。在頻率域的頻譜圖中，影像的低頻成分會集中在中心，而高頻成分則分佈於外圍；至於欲去除的週期性垂直條紋雜訊，則會表現為頻譜圖中心水平軸上特定的高亮點能量集中區。

接著，程式建構了一個與頻譜圖尺寸相同的全通濾波器遮罩，並針對上述雜訊特徵進行頻率濾除。我們將頻譜圖中心點左右兩側特定範圍內的數值強制歸零，這相當於設計了一個針對特定頻段的帶阻濾波器。透過阻斷這些代表週期性條紋的頻率訊號，我們能夠在不破壞文字筆畫的前提下，精準地將干擾條紋的能量移除。完成濾波後，利用反傅立葉變換將處理過的頻譜訊號還原回空間域，並計算震幅大小以重建影像。那為了進一步提升文字的對比度，採用了 Otsu 的Otsu's Binarization，該演算法能自動計算最佳閾值，將灰階影像轉化為黑白分明的二值影像，至此完成了影像的去噪與增強工作。

最後，程式將經過前處理的乾淨影像串接至 Google Cloud Vision API 進行文字辨識。

3) Denoise result:



Original Result

4) **OCR result:**

```
==============================================
          FINAL OCR RESULT
==============================================
andrinity Dix beteren Stockley
f Kny and State of Tennesse
Andrew Jackson of the County
tate aforesaid of the other part
5 and Stockley Donelson for a
of the Sum of two thousand
hand haid the receight whirst
But alien enteroff and and f
hath and by this presents!
Confir
Jackson his heirs a
Certain traits or parcels of La
sand acres on thousandwyre
long f
Belf
==============================================
```

4. text-spotshade.tif

1) **Source code:**

```python
import cv2
import numpy as np
import os
import sys
from google.cloud import vision

current_dir = os.path.dirname(os.path.abspath(__file__))
key_path = os.path.join(current_dir, "key.json")
if os.path.exists(key_path):
    print(f"[INFO] Successfully loaded Service Account Key: {key_path}")
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = key_path
else:
    print(f"[ERROR] Key file not found at: {key_path}")
    sys.exit()

def correct_illumination(image_path):
    print(f"[STEP 1] Processing image: {image_path} ...")
    img = cv2.imread(image_path, 0)
    if img is None:
        print(f"[ERROR] Could not read image file: {image_path}")
        sys.exit()
```

```
    background_illumination = cv2.GaussianBlur(img, (101, 101), 0)
    result_float = cv2.divide(img, background_illumination, scale=255)
    result_uint8 = np.uint8(result_float)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    enhanced_img = clahe.apply(result_uint8)
    _, img_binary = cv2.threshold(enhanced_img, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    output_filename = "cleaned_spotshade_v2.png"
    cv2.imwrite(output_filename, img_binary)
    print(f"[INFO] Illumination correction complete. Saved as:
{output_filename}")
    return output_filename

def recognize_text_google(image_path):
    print(f"[STEP 2] Uploading to Google Cloud Vision API ...")
    client = vision.ImageAnnotatorClient()
    with open(image_path, 'rb') as image_file:
        content = image_file.read()
    image = vision.Image(content=content)
    image_context = vision.ImageContext(language_hints=["en"])
    response = client.document_text_detection(image=image,
image_context=image_context)
    if response.error.message:
        raise Exception(f'API Error: {response.error.message}')
    return response.full_text_annotation.text
if __name__ == "__main__":
    input_image = "text-spotshade.tif"
    if not os.path.exists(input_image):
        if os.path.exists("text-spotshade.jpg"):
            input_image = "text-spotshade.jpg"
        elif os.path.exists("text-spotshade.png"):
            input_image = "text-spotshade.png"
        else:
            print(f"[ERROR] Input file '{input_image}' not found.")
            sys.exit()
    try:
        clean_image_path = correct_illumination(input_image)
        text_result = recognize_text_google(clean_image_path)
```

```
        print("\n==============================================")
        print("          FINAL OCR RESULT (Improved)         ")
        print("==============================================")
        print(text_result)
        print("==============================================")
    except Exception as e:
        print(f"\n[EXCEPTION] An error occurred: {e}")
```

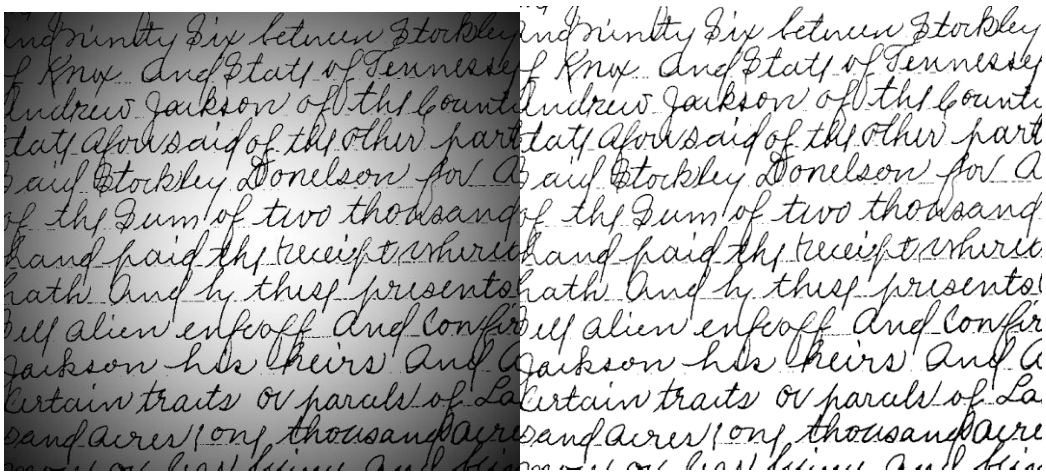
2) Describe method:

在影像前處理階段，目標是移除低頻的光照變化並保留高頻的文字細節。程式首先讀取灰階影像，並利用大尺寸核（Kernel size 101x101）的高斯模糊技術來估測影像的背景光照分布模型。由於光照變化通常較為平緩，而文字筆畫變化劇烈，透過高斯模糊可有效提取出光照層。接著，執行 Background Division運算，將原始影像的像素值除以估測出的背景亮度值。此數學操作能將光照不均的區域進行歸一化，使得原本因陰影而變暗的角落與過亮的中心區域亮度趨於一致，從而數學上消除了聚光燈效應。

為了進一步提升文字的可讀性，採用了限制對比度自適應直方圖等化（CLAHE），針對影像的局部區域增強對比度，避免雜訊過度放大。最後再透過 Otsu 的自適應演算法自動計算最佳閾值進行二值化，將影像轉化為黑白分明、無陰影干擾的清晰圖檔。

影像經過去陰影與增強處理後，進入文字辨識階段。採用利用 Google Cloud Vision API 的文件文字偵測服務。

3) Denoise result:



Original　　Result

4)OCR result:

```
==============================================
          FINAL OCR RESULT
==============================================
andrinity Dix beteren Stockley
f kny And State of Tennesse
Andrew Jackson of the County
tate aforesaid of the other part
3 and stockley Donelson for a
of the Dum of two thousand
hand paid the receight whirst
hath and by these presents!
But alien enfeoff and and
Belf
confir
Jackson his heirs a
Certain traits or parcels of La
•thousand aire
sand
==============================================
```